Name: Rahul Veer **PRN:** 202501100049

Case Study: Student Performance Analysis

Pandas Data Cleaning, Preparation & Analysis

■ Dataset Overview

File	Columns	Description
students_info.csv	ID, Name, Branch	Basic student details
marks.csv	ID, Subject, Marks	Marks per subject per student
attendance.csv	ID, Attendance	Attendance percentage of each student

■■ Section 1: Setup & Data Loading

Step 1: Import Required Libraries

Import pandas for data manipulation and numpy for numerical operations.

```
import pandas as pd
import numpy as np
```

Step 2: Load All Three Datasets into DataFrames

Read each CSV file using **pd.read_csv()** and store as separate DataFrames.

```
students_df   = pd.read_csv('students_info.csv')
marks_df      = pd.read_csv('marks.csv')
attendance_df = pd.read_csv('attendance.csv')
```

Step 3: Display First 5 Rows of Each Dataset

Use **.head()** to preview the top 5 rows of each DataFrame.

```
print(students_df.head())
print(marks_df.head())
print(attendance_df.head())
```

```
Output: Prints top 5 rows showing ID, Name, Branch | ID, Subject, Marks | ID, Attendance
```

■ Section 2: Missing Value Analysis

Step 4: Identify Missing Values in All Datasets

Use **.isnull()** to create a boolean mask showing where values are missing.

```
print(students_df.isnull())
print(marks_df.isnull())
print(attendance_df.isnull())
```

```
Output: Boolean DataFrame – True where value is NaN, False where value exists.
```

Step 5: Count Total Missing Values Column-wise

Chain **.isnull().sum()** to count NaN values per column across each dataset.

```
print("Students missing values:\n",   students_df.isnull().sum())
print("Marks missing values:\n",      marks_df.isnull().sum())
print("Attendance missing values:\n", attendance_df.isnull().sum())
```

```
Output: Each column name with count of missing values, e.g., Attendance: 2
```

Step 6: Handle Missing Values in Attendance Dataset Using Mean

Replace NaN values in the Attendance column with the column's mean value to preserve data integrity.

```
mean_attendance = attendance_df['Attendance'].mean()
attendance_df['Attendance'].fillna(mean_attendance, inplace=True)

print("Missing after fill:", attendance_df['Attendance'].isnull().sum())
```

```
Output: Missing after fill: 0 (all NaN replaced with column mean)
```

■ Section 3: Data Cleaning & Type Conversion

Step 7: Remove Any Duplicate Rows (if present)

Use **.drop_duplicates()** to remove rows that are exact duplicates. **inplace=True** modifies the DataFrame directly.

```
students_df.drop_duplicates(inplace=True)
marks_df.drop_duplicates(inplace=True)
attendance_df.drop_duplicates(inplace=True)

print("Rows after dedup — Students:", len(students_df),
      " Marks:", len(marks_df),
      " Attendance:", len(attendance_df))
```

```
Output: Row counts after removing duplicates (e.g., Rows after dedup — Students: 8)
```

Step 8: Convert Categorical Column 'Branch' into Numerical Format

Use **pd.get_dummies()** (One-Hot Encoding) to convert the Branch column into binary columns — one per unique branch value.

```
students_encoded = pd.get_dummies(students_df, columns=['Branch'])
print(students_encoded.head())

# Alternative — Label Encoding:
students_df['Branch_Code'] = students_df['Branch'].astype('category').cat.codes
print(students_df[['Branch', 'Branch_Code']].head())
```

```
Output: New binary columns like Branch_CS, Branch_IT, Branch_MECH added to DataFrame.
```

Step 9: Ensure All Numeric Columns are of Correct Data Type

Use **.dtypes** to inspect types and **pd.to_numeric()** to coerce any mistyped columns.

```
print(marks_df.dtypes)
print(attendance_df.dtypes)

marks_df['Marks']           = pd.to_numeric(marks_df['Marks'],           errors='coerce')
attendance_df['Attendance'] = pd.to_numeric(attendance_df['Attendance'], errors='coerce')

print("\nAfter conversion:")
print(marks_df.dtypes)
```

```
Output: Marks → float64, Attendance → float64 (errors='coerce' sets invalid values to NaN)
```

■ Section 4: Normalization

Step 10: Perform Normalization on Marks Column

Apply **Min-Max Normalization** to scale the Marks column to a 0–1 range. Formula: **Normalized = (x - min) / (max - min)**

```
min_marks = marks_df['Marks'].min()
max_marks = marks_df['Marks'].max()

marks_df['Marks_Normalized'] = (marks_df['Marks'] - min_marks) / (max_marks - min_marks)

print(marks_df[['Marks', 'Marks_Normalized']].head(8))
```

```
Output: New column Marks_Normalized with values between 0.0 and 1.0.
```

Note: Normalization ensures no single large-value feature dominates ML models.

■ Section 5: Merging Datasets

Step 11: Merge All Three Datasets Using ID as Key

Combine all three DataFrames step-by-step on the common **ID** column using **pd.merge()**.

```
# Step 1: Merge students + marks
merged_df = pd.merge(students_df, marks_df, on='ID')

# Step 2: Merge result + attendance
merged_df = pd.merge(merged_df, attendance_df, on='ID')

print(merged_df.shape)
print(merged_df.head())
```

```
Output: Combined DataFrame with all columns — ID, Name, Branch, Subject, Marks, Attendance.
```

Step 12: Inner Join vs Left Join

Perform both join types to understand how they differ in including unmatched records.

```
# Inner Join — only matching IDs in BOTH DataFrames
inner_df = pd.merge(students_df, marks_df, on='ID', how='inner')
print("Inner Join shape:", inner_df.shape)

# Left Join — all rows from left (students_df), NaN for unmatched right
left_df = pd.merge(students_df, marks_df, on='ID', how='left')
print("Left Join shape:", left_df.shape)
```

Step 13: Difference Between Inner Join and Left Join

Inner Join: Returns only rows where the ID exists in *both* DataFrames. Students without any marks record are excluded from the result.

Left Join: Returns *all rows* from the left DataFrame (students_df). If a student has no marks, their Subject and Marks columns will appear as NaN.

■ Section 6: Pivot Table & Melt

Step 16: Convert Dataset into a Pivot Table

Create a pivot table with **Name** as rows, **Subject** as columns, and **Marks** as values.

```
pivot_df = merged_df.pivot_table(
    index='Name',
    columns='Subject',
    values='Marks',
    aggfunc='mean'
)

pivot_df.columns.name = None   # remove column axis label
pivot_df.reset_index(inplace=True)
print(pivot_df)
```

```
Output: Wide-format table — each student (row) × subject (column) with their marks.
```

Step 17: Convert Pivot Table Back Using melt()

Use **pd.melt()** to unpivot (convert wide → long format). This reverses the pivot operation, restoring Subject and Marks as separate columns.

```
melted_df = pd.melt(
    pivot_df,
    id_vars=['Name'],
    var_name='Subject',
    value_name='Marks'
)

print(melted_df.head(10))
```

```
Output: Long-format table — columns: Name, Subject, Marks (original structure restored).
```

■ Section 7: Analysis & Insights

Step 18: Find Average Marks of Each Student

Use **groupby()** on Name and apply **.mean()** on Marks to find each student's average.

```
avg_marks = merged_df.groupby('Name')['Marks'].mean().reset_index()
avg_marks.columns = ['Name', 'Average_Marks']
avg_marks = avg_marks.sort_values('Average_Marks', ascending=False)
print(avg_marks)
```

```
Output: Sorted table — Name | Average_Marks (highest performing students at top).
```

Step 18b: Find Highest Marks in Each Subject

Group by **Subject** and apply **.max()** to find the top score in each subject.

```
top_marks = merged_df.groupby('Subject')['Marks'].max().reset_index()
top_marks.columns = ['Subject', 'Highest_Marks']
print(top_marks)
```

```
Output: Subject | Highest_Marks — e.g., Mathematics: 98, Physics: 95, etc.
```

Step 18c: Find Student with Highest Attendance

Use **.idxmax()** to find the index of the highest attendance value, then retrieve the student name.

```
idx = attendance_df['Attendance'].idxmax()
top_student_id   = attendance_df.loc[idx, 'ID']
top_student_name = students_df.loc[students_df['ID'] == top_student_id, 'Name'].values[0]
top_attendance   = attendance_df.loc[idx, 'Attendance']

print(f"Student with Highest Attendance: {top_student_name}")
print(f"Attendance: {top_attendance:.1f}%")
```

```
Output: Student with Highest Attendance: [Name] Attendance: 97.0%
```

■ Quick Reference Summary

Step	Operation	Key Method Used
1	Import libraries	`import pandas as pd`
2	Load datasets	`pd.read_csv()`
3	Preview data	`df.head()`
4	Find missing values	`df.isnull()`
5	Count missing column-wise	`df.isnull().sum()`
6	Fill missing with mean	`df.fillna(mean, inplace=True)`
7	Remove duplicates	`df.drop_duplicates()`
8	Encode categorical column	`pd.get_dummies() / .cat.codes`
9	Fix data types	`pd.to_numeric()`
10	Min-Max normalization	`(x - min) / (max - min)`
11	Merge all datasets	`pd.merge(df1, df2, on="ID")`

12	Inner & Left joins	`how="inner" / how="left"`
13	Join difference	`Inner: matched only, Left: all left rows`
16	Pivot table	`df.pivot_table(index, columns, values)`
17	Melt (unpivot)	`pd.melt(id_vars, var_name, value_name)`
18	Avg marks per student	`groupby("Name")["Marks"].mean()`
18b	Highest marks per subject	`groupby("Subject")["Marks"].max()`
18c	Student with max attendance	`attendance_df["Attendance"].idxmax()`